

Objetivo del tema

En este tema aprenderás a comunicar una Actividad con otra, ya sea dentro de la propia App como en otro componente instalado en el dispositivo móvil, como una cámara de fotos.

2.1 El Intent

Antes de meternos con los *intents*, tenemos que comprender lo que es un **componente**.

La versión abreviada es que el propio S.O. de Android y todas las Apps que pueda tener instaladas en un dispositivo concreto son o tienen componentes. Android permite, siguiendo unas reglas, que un componente se comunique con otro componente.

Por ejemplo, si quieres que tu App utilice la cámara de fotos del móvil, no tienes que escribir el código completo o usar una librería externa. Puedes ‘preguntar’ al sistema operativo a ver si existen Apps (componentes) capaces de hacer el trabajo por ti.

Esto es lo que se llama **comunicación entre componentes**, y sirve entre otras cosas, para que tu App pueda funcionar con cualquiera de las cámaras de fotos de cualquier dispositivo móvil imaginable del mercado.

La definición de libro es la siguiente: los **componentes** son los bloques de construcción esenciales de una App. Estos *componentes* están relacionados de forma débil con nuestra App a través del fichero AndroidManifest.xml. Los componentes más importantes que pueden usarse son:

- **Actividades:** Hasta ahora hemos visto Apps con una sola *actividad*, pero pueden tener varias. Las *actividades* pueden ser ejecutadas por otras *actividades* si se siguen los pasos adecuados. Normalmente esto ocurre cuando tienes una App con dos o más ‘pantallas’ y quieres ‘navegar’ de una a otra.
- **Servicios:** Los *servicios* son Apps que se ejecutan sin interfaz, en background. Podríamos decidir que el código para conectarnos a una wifi y mostrar en tiempo real el precio de unas acciones se plantea como un *servicio*.
- **Proveedores de Contenido:** Manipulan una serie de datos que pueden ser compartidos por nuestra App. Por ejemplo, un *proveedor de contenidos* puede ser la lista de contactos de nuestro móvil. Nuestra App podría usarla para crear nuevos contactos, leerlos, etc.
- **Receptores de Broadcast:** Apps que reciben mensajes de multidifusión de otras Apps. Podrían ser por ejemplo Apps que avisan de que te ha llegado tu pedido de Amazon, o que tienes un mail, etc.

Nosotros sólo vamos a trabajar con *actividades*, el resto los veremos más adelante.

Cuando una *actividad* intenta mandar un mensaje a otra *actividad*, a eso se le llama **Intent**.

La comunicación con intents es **asíncrona**, lo que puede dar lugar a problemas de comunicación, sincronización, etc. Los *intents* también pueden usarse para comunicar una *actividad* con un *servicio*, etc.

2.2 El Intent Explícito

Un **Intent Explícito** sucede cuando indicamos qué componente queremos invocar; normalmente otra *actividad* de la misma App. Cuando tengamos que delegar una tarea de una *actividad* a otra *actividad* recurrimos al **intent**.

En nuestro proyecto, para agregar una *actividad* secundaria vamos al explorador de proyectos, botón derecho, **New-> Activity-> EmptyActivity**. Esto crea una clase Kotlin, un layout y además **registra** esa nueva *actividad* en el **AndroidManifest.xml**.

Nunca hagas una actividad nueva a mano para evitar problemas.

Vamos a construir un ejemplo en el que una *actividad* realiza un intent sobre otra *actividad*. Creamos un botón con un evento que lanza el intent, y usamos el método **startActivity ()** para iniciar esa nueva *actividad*. Si implementas un ejemplo sencillo, verás que al darle al darle al botón la App cambiará de una ventana a otra.

Pues bien: no te dejes **engañar** por lo que ves. Lo que realmente ocurre es lo siguiente:

1. Se inicializa la nueva *actividad*, ejecutándose su **onCreate ()**.
2. La *actividad* original se va a **la pila**. Ahora tenemos dos *actividades* compitiendo por recursos.
3. Si en algún momento la nueva *actividad* hace **finish ()**, o el usuario pulsa al botón de atrás del móvil; entonces la nueva *actividad* se cierra y se saca de la pila.
4. Se vuelve a mostrar la *actividad* original.

Date cuenta de que cada vez que haces un *intent* estás inicializando una nueva actividad. Es posible que creas que para 'volver' a la *actividad* anterior basta con hacer otro Intent: pero esto es un **error**.

Mira lo que sucede si intentas hacer eso:

1. Estamos en *actividad1*. Haces un intent a la *actividad2*.
2. Se inicializa la *actividad2*, ejecutándose su **onCreate ()**.
3. La *actividad1* se va a **la pila**. Ahora tenemos dos *actividades* compitiendo por recursos.
4. En la *actividad2*, haces un intent a la *actividad1* creyendo que eso es 'volver'.
5. La *actividad2* se va a **la pila**. Ahora tenemos tres *actividades* compitiendo por recursos, de las cuales tienes una copia de *actividad1* y dos copias de *actividad2*.
6. Se muestra la segunda *actividad2* recién creada.

¡Enhorabuena! No solo estás consumiendo memoria innecesariamente, sino que al duplicar *actividades* no tienes ni idea de lo que estás haciendo. La forma correcta de 'volver' a una actividad 'anterior' es **finish ()**, que finaliza la actividad en curso. Al hacerlo, automáticamente se vuelve a la siguiente *actividad* en la pila. Ten en cuenta también que al hacer **finish ()** sobre una *actividad* ejecuta automáticamente su método **onDestroy ()**

2.3 Paso de información a la subactividad

Una *actividad2* iniciada mediante **startActivity ()** es independiente de la *actividad1* que la lanzó originalmente. En ocasiones, será necesario proporcionar a esa *actividad2* algún tipo de información:

Paso 1 – Intent: Definimos el Intent

```
buttonOk.setOnClickListener {
    val intent = Intent(this, Activity2::class.java)
    startActivity(intent)
}
```

Paso 2 – Pasamos los parámetros: Añadimos parámetros mediante **putExtra ()**.

```
buttonOk.setOnClickListener {
    val intent = Intent(this, Activity2::class.java)
    intent.putExtra("Veces", veces)
    startActivity(intent)
}
```

Paso 3 – Recogemos los parámetros: Esta información puede ser recogida en el onCreate de *actividad2*

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_second)

    // Recogemos el dato "Veces" enviado desde Activity1
    val nombre = intent.getStringExtra("Veces")
}
```

2.4 Retorno de resultados desde la subactividad

También podemos enviar resultados de vuelta a la *actividad1* en cuanto la *actividad2* finaliza. En este caso, necesitaremos usar la **Activity Result API**. Lo que hacemos es registrar la *actividad2* y preparar un **callback** para tratar sus respuestas en la *actividad1*.

Paso 1 – Callback: Añadimos un callback al Intent, con lo que preparamos a la *actividad1* para que responda de una u otra forma a las respuestas de la *actividad2*

```
buttonOk.setOnClickListener {
    val getResult = registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
        if (result.resultCode == Activity.RESULT_OK) {
            // Si la actividad2 finaliza con éxito
            val veces = result.data?.getIntExtra("veces", 0) ?: 0
            Toast.makeText(this, "Valor recibido: $veces", Toast.LENGTH_SHORT).show()
        } else {
            // Si no lo hace
            Toast.makeText(this, "Operación cancelada", Toast.LENGTH_SHORT).show()
        }
    }

    // Lanzar la Activity secundaria
    val intent = Intent(this, Activity2::class.java)
    getResult.launch(intent)
}
```

Paso 2 – Enviar el resultado: Una vez preparada la *actividad1* para tratar las posibles respuestas de la *actividad2*, esta actividad secundaria puede ahora responder como más le guste:

```
buttonOk.setOnClickListener {
    val intent = Intent()
    intent.putExtra("Veces", veces)
    setResult(Activity.RESULT_OK, intent)
    finish()
}

buttonCancel.setOnClickListener {
    setResult(Activity.RESULT_CANCELED)
    finish()
}
```

Ejercicio 16

Crea una App similar a la mostrada en la imagen.



Tiene dos *actividades* diferentes. En la MainActivity se solicita que presiones un botón para iniciar la encuesta. En esta actividad también se mostrará el resultado de esta. La ProvinciaActivity muestra un RadioGroup con diferentes provincias para seleccionar. Cuando se seleccione una provincia y se le de a aceptar, la provincia seleccionada debe ser enviada a la MainActivity.

Esta práctica está solucionada en **P7 – IntentExplícito**

Ejercicio 17

Crea un minijuego de temática friki (o de lo que te guste). Prepara diez preguntas sobre el tema con múltiples respuestas. Por supuesto, cada pregunta está en una actividad diferente. El usuario tiene que contestar, y se controla si ha respondido bien. Tienes que ir pasando de una pantalla a otra la puntuación de respuestas correctas. La última pantalla deberá mostrar cuántas respuestas correctas has conseguido. Otro botón me permitirá reiniciar el juego.

2.5 El Intent Implícito

Como ya se ha comentado, existe otra forma de iniciar una *actividad*: un intent **implícito**. En este caso, se solicita que un componente anónimo de una App sin determinar se encargue de realizar el trabajo. Esto es similar a colocar un aviso en un tablón de anuncios. Escribes qué quieres hacer y la información necesaria para hacerlo, y *alguien responderá* a ese anuncio. Date cuenta que no sabes con antelación quién se encargará de responder al aviso.

Para poder hacer un *intent implícito* necesitas indicar la **acción** que quieres realizar y pasarle la **URI** con los datos que necesita. Si es necesario, puedes enviar más información como un **extra**.

Existen muchas **acciones** diferentes y están catalogadas. Es posible preparar una App para que responda a una o más acciones; además, puede que varias Apps sean capaces de responder a la misma acción. Por norma general, es el Sistema Operativo el que elige qué App es la más adecuada para responder a una acción, excepto si queremos darle la opción al usuario de elegir qué App quiere utilizar mediante un **chooser**.

Las acciones más comunes son:

Estados	
ACTION_ANSWER	Abre la app que maneja las llamadas de teléfono entrantes
ACTION_CALL	Muestra el marcador de teléfonos e inmediatamente inicia una llamada al número indicado por la URI.
ACTION_DELETE	Inicia una actividad para eliminar los datos referenciados por la URI del intent
ACTION_DIAL	Muestra el marcador de teléfonos con un número premarcado que es el pasado por la URI
ACTION_EDIT	Inicia una actividad para editar los datos referenciados por la URI del intent
ACTION_INSERT	Inicia una actividad para insertar nuevos datos en el Cursor referenciados por la URI del intent
ACTION_PICK	Lanza una actividad para escoger un elemento del proveedor de contenidos especificado por la URI del intent
ACTION_SEARCH	Inicia una actividad de búsqueda
ACTION_SENDTO	Inicia una actividad para enviar un mensaje al contacto especificado por la URI del intent.
ACTION_SEND	Inicia una actividad para enviar una serie de datos mediante el intent.
ACTION_VIEW	Es la acción más común. Solicita que los datos referenciados por URI del intent sean mostrados de la forma más adecuada posible. q

En cualquier caso, tocará **investigar por internet** exactamente qué acción y cómo se pasan parámetros para cada cosa que queramos hacer. No es lo mismo solicitar el uso de la cámara de fotos que abrir un vídeo de música en YouTube en nuestro navegador predeterminado.

En el siguiente ejemplo solicitamos que ‘alguien’ nos muestre una dirección web. Si iniciásemos el *intent*, el sistema abriría el navegador predeterminado. En cambio, al crear un *chooser* a partir del *intent*, el sistema nos mostraría un pop-up con todas las Apps capaces de responder a la acción ACTION_VIEW y nos dejaría escoger una.

```
val intent = Intent ( Intent.ACTION_VIEW, Uri.parse( URL ) );
val chooser = intent.createChooser(intent, getText( R.string.txt_intent_web ));

if ((chooser.resolveActivity(getPackageManager())) != null) {
    startActivity(chooser);
    Toast.makeText(this, getText( R.string.txt_ok ), Toast.LENGTH_LONG).show();
} else {
    Toast.makeText(this, getText( R.string.txt_error_2 ), Toast.LENGTH_LONG).show();
}
}
```

Paso 1 – Intent: Declaramos un Intent, solicitando ver un vídeo de YouTube (URL)

Paso 2 – Chooser: En tiempo de ejecución, se mostrará un pop-up con todas las Apps capaces de mostrarnos YouTube. Esto es configurable para cada dispositivo móvil y las Apps que tenga instaladas.

2.6 Permisos

Para poder utilizar la cámara, internet o cualquier componente tienes que darle permisos en el **Manifest**. Diferentes componentes necesitarán diferentes permisos, obviamente. En el caso de la cámara, indicamos que el atributo required es true. Esto entre otras cosas hace que Google Play no permita que dispositivos sin cámara puedan descargar nuestra App.

```
<uses-feature
    android:name="android.hardware.camera"
    android:required="true" />
```

Es nuestra responsabilidad comprobar la disponibilidad de la cámara en tiempo de ejecución. Si no disponemos de ella, habrá que deshabilitar las partes de nuestro código que dependan de la cámara. Podemos hacerlo con:

```
PackageManager packageManager = getPackageManager ();
boolean cameraAvailable = packageManager.hasSystemFeature(Pack-ageManager.FEATURE_CAMERA_ANY);
```

2.6 Retorno de resultados desde la subactividad

Al igual que sucede con los Intent Explícitos, el componente que responde a un Intent Implícito puede responderte con información. Si queremos usar la cámara, es obvio que nos devolverá la foto que hemos tomado. El funcionamiento es similar. Siguiendo esta idea:

Paso 1 – Callback: Registramos el callback con el tratamiento de los datos enviados desde la cámara. De nuevo, para otros componentes, será necesario investigar cómo los devuelve:

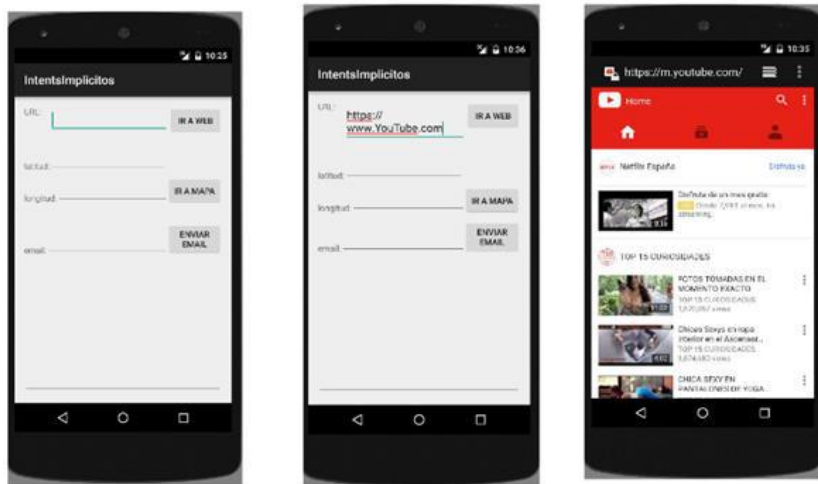
```
private val cameraLauncher = registerForActivityResult(  
    ActivityResultContracts.StartActivityForResult()  
) { result ->  
    if (result.resultCode == RESULT_OK) {  
        val extras = result.data?.extras  
        val imageBitmap = extras?.get("data") as Bitmap  
        imageView.setImageBitmap(imageBitmap)  
    }  
}
```

Paso 2 – Intent: Hacemos el intent.

```
boton.setOnClickListener {  
    val intent = android.content.Intent(MediaStore.ACTION_IMAGE_CAPTURE)  
    cameraLauncher.launch(intent)  
}
```

Ejercicio 18

Crea una App similar a la mostrada en la imagen.



Esta práctica consiste en una *actividad* con cuatro **EditText** y tres **Buttons**.

- Cuando se pulse el **Button** “Ir a Web”, se cogerá la URL y se realizará un Intent Implícito para que se abra la página web indicada. No te olvides de poner `https://` o no funcionará.
- Cuando se pulse el **Button** “Ir a Mapa”, se cogerá la longitud y latitud y se realizará un Intent Implícito para que se abra el Google Maps y nos enseñe la localización indicada. La longitud y latitud de Bilbao son: 43.26271 y -2.92528
- Cuando se pulse el **Button** “Enviar email”, se cogerá la dirección de correo indicada y se realizará un Intent Implícito para que se envíe un correo.

No te olvides de dar permisos de acceso a Internet en el Manifest.

Nota: Si bien las dos primeras partes no dan mayores problemas, el Intent al correo puede que lo programes bien, pero haga cosas raras. Eso es porque no tenemos ni idea de qué componentes del móvil van a salir de candidatos para que envíes el mail, y cada uno funciona a su bola. Si eliges **Gmail**, la App te obligará a que le des una cuenta de usuario válida y una clave para vincularla al móvil. Cuando termines, entonces verás que se abre el Gmail y te muestra automáticamente un correo redactado con los datos que le has pasado. Sólo falta que le des a ‘enviar’. No tienes por qué configurar nada si no quieres.

Ejercicio 19

Realiza una App que sea capaz de acceder a la agenda del móvil y cargue una lista con los nombres y números de teléfono. Al pulsar a un número de la lista, la App deberá de mostrar el marcador de teléfonos con el número ya marcado. (No llares directamente al número)

Ejercicio 20

Diseña una App que sea capaz de tomar fotos y mostrarlas después en tu Actividad. El Android Studio permite que puedas probar la cámara mediante el emulador, dado que 'simula' una habitación virtual. Investiga un poco su funcionamiento.

Ejercicio 21

Diseña una App que sea capaz de grabar vídeo fotos y mostrarlas después en tu Actividad. El Android Studio permite que puedas probar la cámara mediante el emulador, dado que 'simula' una habitación virtual. Investiga un poco su funcionamiento.

2.7 Nuestra Actividad también responde a Intents

Ya hemos visto que es posible realizar solicitudes de una determinada acción al sistema: el *intent implícito*. Pero también es posible que sea nuestra App la que pueda dar ese servicio y, por tanto, sea elegible por **chooser**. Para ello, primero tenemos que registrar nuestra App como manejadora potencial de un determinado tipo de *intent*. Y todo eso se hace a través del fichero.

AndroidManifest.xml de nuestra App. Al añadir un intent-filter estaremos anunciando al sistema Android que nuestra App es capaz de responder a ciertas **acciones**.

```
<activity
    android:name=".Activity2"
    android:exported="true">

    <intent-filter>
        <action android:name="es.ua.eps.intent.action.HAZ_ALGO" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.SELECTED_ALTERNATIVE" />
        <data android:mimeType = "vnd.myApp.cursor.item/*" />
    </intent-filter>

    <meta-data
        android:name="android.app.lib_name"
        android:value="" />
</activity>
```

Cada intent-filter consta de los siguientes elementos:

- **Action:** especifica el nombre de la acción o acciones que la actividad es capaz de gestionar. Es posible utilizar el nombre de una acción que ya existe, o inventarte uno. Obviamente, el nombre debería ser largo y descriptivo, marcando la ruta de paquetes.
- **Category:** especifica bajo qué circunstancias la App atenderá a la petición. Se pueden indicar varias. Cuando se lance un *intent implícito* para que lo atienda tu App, se le añadirán las categorías que deseemos, y solamente responderán las Apps que contengan todas las categorías indicadas.
- **Data:** indica sobre qué tipos de datos actuará nuestra *actividad*. Es posible indicar varios. Es habitual usar una URI para identificar el dato de forma implícita, aunque podemos usar también un tipo MIME.

Es posible añadir una categoría a un intent. Recuerda que únicamente responderán al intent aquellas Apps cuyas categorías contengan todas las categorías indicadas.

- **ALTERNATIVE:** Nuestra app se presenta como alternativa, aunque quien vaya a hacer la llamada no conozca la acción que realiza. Útil cuando tenemos acciones propias que no son los estándares de Android y que otras apps no conocerán.
- **SELECTED_ALTERNATIVE:** Igual que la anterior, pero cuando se le presente al usuario la app a elegir, la nuestra aparecerá seleccionada por defecto.
- **BROWSABLE:** La acción está disponible para ser lanzada sobre un navegador web. Cuando un *intent* se lance desde un navegador, siempre incluirá esta categoría, luego solo encontrará apps BROWSABLE.
- **DEFAULT:** Esta acción siempre se añade por defecto al realizar un *intent*.
- **GADGET:** Nuestra *actividad* puede ser ejecutada empotrada dentro de otra *actividad*.
- **HOME:** Si no especificamos acción, esta actividad es una alternativa a la pantalla de inicio nativa.
- **LAUNCHER:** Nuestra app aparecerá en el lanzador de aplicación de Android. Esta categoría aparece por defecto en la *actividad principal* de nuestro proyecto.

A modo de ejemplo:

```
buttonOk.setOnClickListener {
    val intent = Intent(this, Activity2::class.java)
    intent.addCategory(Intent.CATEGORY_DEFAULT)
    startActivity(intent)
}
```

Finalmente, en los intent implícitos, el tipo de dato se especifica mediante una URI.

```
val viewIntent = Intent (Intent.ACTION_VIEW)
viewIntent.data = Uri.parse ( uriString: "http://www.google.com");
```

A partir de esa URI el sistema infiere el tipo MIME de los datos, aunque también podríamos indicárselo explícitamente:

```
val viewIntent = Intent (Intent.ACTION_VIEW)
viewIntent.setType("text/html")
```